牛客周赛 Round 122 题解

A.ICPC Problems

模拟即可。

```
void solve(){
    ll n = read();
    for(ll i=0;i<n;i++){
        putchar((char)(65+i)), putchar(' ');
    }
}
```

~~貌似是香港的榜单~~

B.Chess

手玩一下可以发现像题目中一样开始放在 $(1,1)$ 最优。

所有可以到达的格点满足 $(4k_1+1,4k_2+1)$ 或者 $(4k_1+3,4k_2+3)$ 。

则有 $(\lfloor\frac{n-1}{4}\rfloor+1)\times(\lfloor\frac{m-1}{4}\rfloor+1)+(\lfloor\frac{n-3}{4}\rfloor+1)\times(\lfloor\frac{m-3}{4}\rfloor+1)$ 个。

特判 $min(n,m)<=2$ 的情况，此时棋子无法移动。

```
void solve(){
    ll n = read(), m = read();
    if(min(n, m)<=2){
        print(1);
    }
    else{
        print(((n-1)/4+1) * ((m-1)/4+1) + ((n-3)/4+1) * ((m-3)/4+1));
    }
}
```

C.Sequence Cost

结论:要不操作一次 l,r, 要不不操作

手玩一下可以发现，必须操作的情况下相当于把最大值降低。

这个过程中，降到原数组的最小值不劣。

则只需比较不操作是否更优即可。

```
void solve(){
    ll n = read();
    vector<ll> a(n);
    for(ll i=0;i<n;i++){
        a[i] = read();
    }
    print(min(accumulate(all(a), 0ll),
*min_element(all(a))*n+*max_element(all(a))));
}
```

D.Digital Deletion

首先考虑如何求 S 的 MEX。

对 a 升序排序后，考虑 a_i。设 $sum = \sum_{i=1}^{i-1} a_i$。

- $i = 0$ 时，显然能构成 $[0, 0]$ （即$[0, sum]$) 的所有数字。
- $i \neq 0$ 时，假设前面的数字能构成 $[0, sum]$ 的所有数字，原先的每一种方案，我们对其加 a_i 后，可以将值域扩展到 $[a_i, sum + a_i]$。

  因此若 $a_i <= sum + 1$,则可以构成完整的值域 ,mex 可以继续扩大。否则不能。

知道了 MEX 后，我们可以枚举每个 i，用 pq 选择最大的满足 $a_i <= cur + 1$ 的最大 a_i。即使用更少的数字去凑 MEX。

注意到 $0 \leq a_i \leq 10^9$，而 0 显然可以直接去掉，记得特判

```
void solve(){
    ll n = read();
    vector<ll> a(n);
    for(ll i=0;i<n;i++) a[i] = read();
    sort(all(a));
    ll sum = 0;
    for(auto v:a){
        if(v <= sum + 1) sum += v;
        else break;
    }
    priority_queue<ll> pq;
    ll cur = 0, use  = 0, idx = 0;
    while(cur< sum){
        while(idx < n && a[idx] <= cur + 1){
            pq.push(a[idx]);
            idx++;
        }
        ll t = pq.top();
        pq.pop();
        cur += t;
        use++;
    }
```

```
    print(n-use);
}
```

E.Block Array

显然最多有 n 个区间，且每个点只会成为一个区间的端点。

首先可以处理出所有的区间。

令 dp_i 为以 i 为右端点的区间的方案数。

对于区间 $[l, r]$，可以选择是否和以 $l-1$ 为有右端点的区间合并，有$dp_r = dp_{l-1} + 1$。

因此对区间按右端点排序后跑 dp 即可。

```
void solve(){
    ll n = read();
    vector<ll> a(n+1), dp(n+1);
    for(ll i=1;i<=n;i++){
        a[i] = read();
    }
    vector<array<ll, 3>> segs;
    vector<ll> rs(n+1, -1);
    for(ll i=1;i<=n;i++){
        if(a[i]!=a[i-1]) segs.pb({a[i], i, i});
        else segs.back()[2]++;
    }
    for(auto [v, l, r]:segs){
        for(ll j=l;j+v-1<=r;j++){
            rs[j+v-1] = j;
        }
    }
    for(ll i=1;i<=n;i++){
        if(rs[i]!=-1){
            dp[i] = dp[rs[i]-1] + 1;
        }
    }
    print(accumulate(all(dp), 0ll));
}
```

F.Sequence Covering

不难想到要让字典序最大，肯定要让越左边的元素越大越好。

对于 i，我们可以选择把 $[i, i+k]$ 里的最大值挪过来，且多个最大值优先选择 idx 小的；或者选择上一步的最大值，即 ans_{i-1} 。

由于每次操作的区间不重叠，实际上不需要修改，只需要维护静态区间最大值。（今天的每日一题喵）

为了优先选择 idx 小的，st 表中实际存储的是元素的下标。

鞭尸 15 分做法:（没有考虑选择上一步的最大值）

样例:

```
1
6 4
-3 1 -4 1 -4 -2
```

```
const ll MAXN = 2e5+9;
ll lg[MAXN];
ll dp[MAXN][20];
ll a[MAXN];
void init(){
    lg[1] = 0;
    for(ll i=2;i<MAXN;i++){
        lg[i] = lg[i>>1]+1;
    }
}

void solve(){
    ll n = read(), k = read();
    for(ll i=1;i<=n;i++){
        a[i] = read();
        dp[i][0] = i;
    }
    function<ll(ll, ll)> cmp = [&](ll l, ll r){
        if(a[l]!=a[r]){
            return a[l] > a[r] ? l : r;
        }
        return min(l, r);
    };
    for(ll k=1;k<=lg[n];k++){
        for(ll s=1;s+(1<<k)<=n+1;s++){
            dp[s][k] = cmp(dp[s][k-1], dp[s+(1<<(k-1))][k-1]);

        }
    }
    function<ll(ll, ll)> qry = [&](ll l, ll r){
        ll k = lg[r-l+1];
        return cmp(dp[l][k], dp[r-(1<<k)+1][k]);
    };
    ll l = 1;
    vector<ll> ans;
    while(l <= n){
        ll r = min(n, l+k);
        ll idx = qry(l, r);

        if(ans.size()&&k&&ans.back()>a[idx]){
            k--;
            l++;
```

```
            ans.pb(ans.back());
        }
        else{
            k -= idx - l;
            for(ll j=0;j<idx-l+1;j++) ans.pb(a[idx]);
            l = idx + 1;
        }


    }
    print(ans);
}
```

c++火车头

```
// FZANOTFOUND
#include <bits/stdc++.h>
using namespace std;

#define pb push_back
#define eb emplace_back
#define fi first
#define se second
#define ne " -> "
#define sep "======"
#define fastio ios::sync_with_stdio(false);cin.tie(0);
#define all(a) a.begin(), a.end()

typedef long long ll;
typedef unsigned long long ull;
typedef long double db;
typedef pair<long long,long long> PLL;
typedef tuple<ll,ll,ll> TLLL;
const ll INF = (ll)2e18+9;
//const ll MOD = 1000000007;
const ll MOD = 998244353;
const db PI = 3.14159265358979323;

//io functions
inline void rd(ll &x){x=0;short f=1;char c=getchar();while((c<'0'||c>'9')&&c!='-
') c=getchar();if(c=='-') f=-1,c=getchar();while(c>='0'&&c<='9') x=x*10+c-
'0',c=getchar();x*=f;}
inline ll read(){ll x=0;short f=1;char c=getchar();while((c<'0'||c>'9')&&c!='-')
c=getchar();if(c=='-') f=-1,c=getchar();while(c>='0'&&c<='9') x=x*10+c-
'0',c=getchar();x*=f;return x;}
inline void pt(ll x){if(x<0) putchar('-'),x=-x;if(x>9)
pt(x/10);putchar(x%10+'0');}
inline void print(ll x){pt(x), puts("");}
inline void print(PLL x){pt(x.fi), putchar(' '), pt(x.se), putchar('\n');}
inline void print(vector<ll> &vec){for(const auto t:vec)pt(t),putchar('
');puts("");}
```

```
inline void print(const map<ll, ll>& g) {for(const auto& [key, value]:g)
{cout<<"key: "<<key<<ne<<value<<" ";}puts("");}
inline void print(vector<PLL> &vec){puts(sep);for(const auto v:vec)
{print(v);}puts(sep);}
inline void print(const map<ll, vector<ll>>& g) {for (const auto& [key, value] :
g) { cout << "key: " << key << ne;for (const auto& v : value) {cout << v << "
";}cout << endl;}}

//fast pow
ll ksm(ll a, ll b=MOD-2, ll M=MOD){a%=M;ll res=1;while(b){if(b&1){res=
(res*a)%M;}a=(a*a)%M;b>>=1;}return res;}

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());//rng()
ull randint(ull l, ull r){uniform_int_distribution<unsigned long long> dist(l,
r);return dist(rng);}


void init(){

}

void solve(){

}

int main(){
    init();
    ll t = 1;
    t = read();
    while(t--){
        solve();
    }
}
```